



Övning 6

Algoritmkonstruktion

DD2350 Algoritmer, datastrukturer och komplexitet ■ HT 2026

Marcus Dicander

Uppgifter: Viggo Kann. Bilder och delar av lösningarna står på tidigare övningsassistenters axlar:
Anton Grensjö, Lisa Li och Alice Heavy.

Dagens tema

Uppgift 1: Inuti eller utanför?

Uppgift 2: Sortering av små heltal

Labb 3: teoriredovisning

Uppgift 3: Hitta det saknade talet

Uppgift 4: Komplex multiplikation

Uppgift 5: Binärträd med speglad struktur

Uppgift 6: Partyproblemet

Avslutning

Sex uppgifter, sex omöjliga krav

Uppgift	Kravet som ser omöjligt ut
1 Inuti eller utanför?	$O(\log n)$: vi hinner inte ens titta på alla hörn
2 Sortering av små heltal	$O(n)$: under den berömda gränsen $\Omega(n \log n)$
3 Hitta det saknade talet	32 bitar och konstant minne: summan får inte plats
4 Komplex multiplikation	tre multiplikationer, där formeln har fyra
5 Speglande binärträd	två träd på en gång, med dekomposition
6 Partyproblemet	linjär tid, fast varje nej kan vända andra ja

Varje krav ser omöjligt ut tills man läser resten av meningen. Idag letar vi efter var i uppgiftstexten tricket gömmer sig. Ledtrådarna fyller vi i på sista bilden.

Uppgift 1: Inuti eller utanför?

Låt P vara en konvex n -hörnig polygon beskriven som en array av hörnen $p_1, p_2, \dots, p_n$ i cyklisk ordning. Konstruera en algoritm som talar om ifall en given punkt q är inuti P . Algoritmen ska gå i tid $O(\log n)$ i värsta fallet.

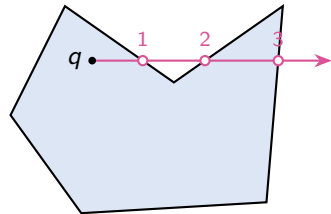
Ledning

I vilka typer av algoritmer brukar vi få logaritmisk tidskomplexitet?

$O(\log n)$ betyder att vi inte ens hinner titta på alla hörn. För en polygon med en miljon hörn tittar vi på ett tjugotal. Resten får vi lita på.

Uppgift 1: första idén är en stråle

- Dra en stråle från q och räkna hur många kanter den skär.
- Udda antal: inuti. Jämnt antal: utanför.
- Fungerar för alla polygoner, även icke-konvexa.
- Problemet: strålen måste provas mot alla n kanter. $O(n)$.



tre skärningar: udda, alltså inuti

Historia

Att strålen fungerar vilar på Jordans kurvsats (Camille Jordan, 1887): en enkel sluten kurva delar planet i ett inre och ett yttre område. Satsen är "uppenbar", men det första allmänt accepterade beviset kom med Oswald Veblen 1905. Som algoritm publicerades strålen av M. Shmrat i CACM 1962.

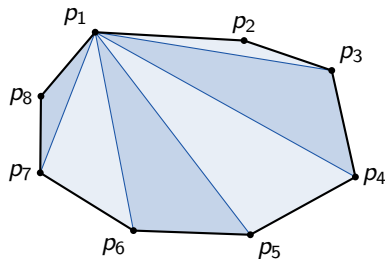
Uppgift 1: sorterat, fast det inte ser ut så

Vanligt missförstånd

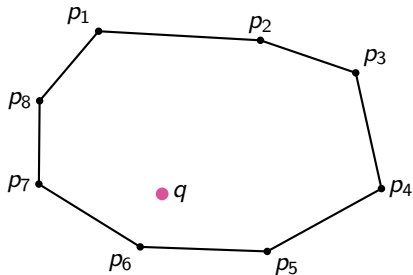
"Binärsökning kräver sorterade data, och hörnen är inte sorterade."

Men de är det. Sett från p_1 ligger $p_2, p_3, \dots, p_n$ i vinkelordning: diagonalerna från p_1 delar P i $n - 2$ trianglar som ligger på rad, som bladen i en solfjäder. Det är precis vad konvex betyder här.

- Diagonalen $p_1 p_{mid}$ delar de återstående trianglarna i två halv.
- Ett teckentest avgör vilken halva q ligger i. Släng den andra.

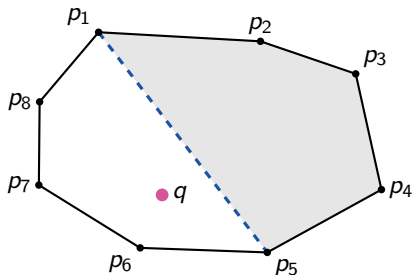


Uppgift 1: binärsökning i solfjädern



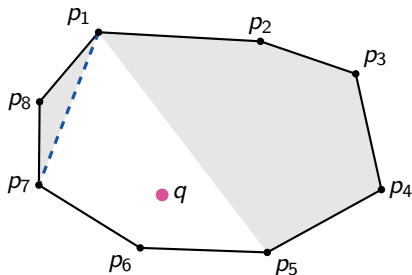
- $InsideConvex(P, q, 2, 8)$

Uppgift 1: binärsökning i solfjädern



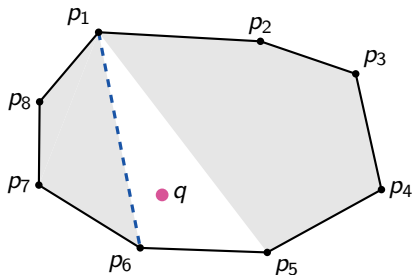
- $InsideConvex(P, q, 2, 8)$
- $mid = 5$: q och p_6 på samma sida om p_1p_5 . Behåll $(5, 8)$.

Uppgift 1: binärsökning i solfjädern



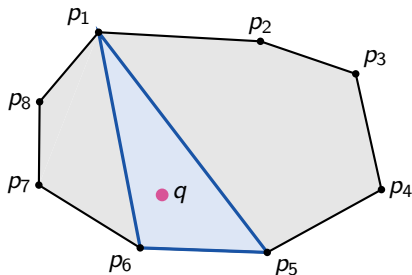
- $InsideConvex(P, q, 2, 8)$
- $mid = 5$: q och p_6 på samma sida om p_1p_5 . Behåll $(5, 8)$.
- $mid = 7$: q och p_8 på olika sidor om p_1p_7 . Behåll $(5, 7)$.

Uppgift 1: binärsökning i solfjädern



- $InsideConvex(P, q, 2, 8)$
- $mid = 5$: q och p_6 på samma sida om p_1p_5 . Behåll $(5, 8)$.
- $mid = 7$: q och p_8 på olika sidor om p_1p_7 . Behåll $(5, 7)$.
- $mid = 6$: q och p_7 på olika sidor om p_1p_6 . Behåll $(5, 6)$.

Uppgift 1: binärsökning i solfjädern



- $InsideConvex(P, q, 2, 8)$
- $mid = 5$: q och p_6 på samma sida om p_1p_5 . Behåll $(5, 8)$.
- $mid = 7$: q och p_8 på olika sidor om p_1p_7 . Behåll $(5, 7)$.
- $mid = 6$: q och p_7 på olika sidor om p_1p_6 . Behåll $(5, 6)$.
- $u = l + 1$: en triangel. q ligger i $p_1p_5p_6$, alltså i P .

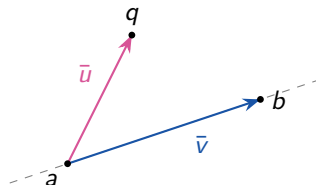
Tre halveringar och ett triangeltest för $n = 8$.

Uppgift 1: vilken sida om linjen?

- Låt $\vec{v} = b - a$ gå längs linjen och $\vec{u} = q - a$ från linjen till punkten.
- Båda ligger i tavlans plan, så $\vec{v} \times \vec{u}$ pekar rakt in i eller rakt ut ur tavlan: bara z-komponenten är kvar,

$$(\vec{v} \times \vec{u})_z = v_x u_y - v_y u_x.$$

- Tecknet avgör. Positivt: q ligger till vänster om $a \rightarrow b$. Negativt: till höger. Noll: på linjen.



$$3 \cdot 2 - 1 \cdot 1 = 5 > 0:$$

q ligger till vänster

Samma sida som p_{mid+1} ?

Samma tecken för q och för p_{mid+1} , relativt linjen $p_1 p_{mid}$. Eftersom vi jämför med ett hörn behöver vi aldrig veta om hörnen går medurs eller moturs.

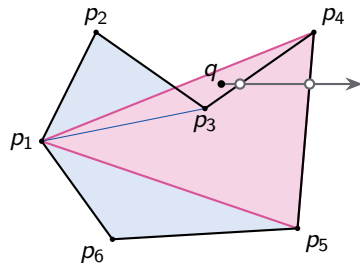
Uppgift 1: algoritmen

```
InsideConvex(P, q, l, u) =  
    if u = l + 1 then                                     /* en triangel */  
        välj en punkt  $q'$  utanför triangeln  $p_l-p_l-p_u$   
        if linjen  $q-q'$  skär exakt en av kanterna i triangeln then  
            return inuti  
        else  
            return utanför  
    else  
        mid  $\leftarrow \lceil (l + u)/2 \rceil$   
        if q är på samma sida om linjen  $p_l-p_{mid}$  som  $p_{mid+1}$  then  
            return InsideConvex(P, q, mid, u)  
        else  
            return InsideConvex(P, q, l, mid)
```

Anropas med $InsideConvex(P, q, 2, n)$. Konstant arbete per anrop och halverat intervall: $T(n) = T(n/2) + c$, alltså $c \log n \in O(\log n)$. Basfallet är strålen igen, på tre kanter; tre teckentester med kryssprodukten går lika bra.

Uppgift 1: vad händer om ...?

- ... q **ligger utanför** P ? Inget särskilt. Sökningen landar ändå i en triangel, och triangeltestet svarar utanför.
- ... q **ligger på en kant**? Kryssprodukten blir 0. Bestäm i förväg om randen räknas som inuti.
- ... **vi har m frågor mot samma polygon?** Strålen: $O(mn)$. Solfjädern: $O(m \log n)$. Här betalar sig logaritmen.
- ... **det tar $O(n)$ att läsa in polygonen?** Ja. Modellen antar att arrayen redan finns; logaritmen gäller frågan.
- ... P **inte är konvex?** Då finns ingen vinkelordning att binärsöka i. Samma punkt, två algoritmer, två svar:



Solfjädern landar i $p_1 p_4 p_5$ och svarar inuti.
Strålen skär två kanter och svarar utanför.

Strålen har rätt.

Uppgift 1: släktingar och historia

Samma idé på andra ställen

- Binärsökning i en sorterad array: samma halvering, samma $T(n) = T(n/2) + O(1)$.
- Bisektionsmetoden för nollställen: behåll halvan där tecknet byter.
- Grahams algoritm för konvexa höljet (1972) vilar på samma teckentest.

Historia

Den första binärsökningen publicerades 1946, den första som fungerade för alla n först 1962 (enligt Jon Bentley). År 2006 visade Joshua Bloch att Javas binärsökning burit på en bugg i ungefär nio år: $(l + u)/2$ svämmar över när arrayen har minst 2^{30} element. Håll kvar den tanken till uppgift 3.

Tricket

Konvex $\Rightarrow$ hörnen ligger sorterade efter vinkel runt $p_1 \Rightarrow$ binärsökning.

Uppgift 2: Sortering av små heltal

Konstruera en algoritm som sorterar n heltal som alla ligger i intervallet $[1..n^3]$ i tid $O(n)$ med enhetskostnad. Tips: tänk på räknasortering och radixsortering.

Vanligt missförstånd

"Sortering kan aldrig gå fortare än $\Omega(n \log n)$."

Gränsen gäller **jämförelsebaserad** sortering: det finns $n!$ möjliga ordningar och varje jämförelse har två utfall, alltså behövs minst $\log_2 n! \in \Theta(n \log n)$ jämförelser i värsta fall. Här vet vi mer: att det är heltal, och hur stora de är. Vi tänker inte jämföra två tal med varandra en enda gång.

Den undre gränsen gäller fortfarande. Den gäller bara inte oss.

Uppgift 2: två genvägar som inte håller

Vanligt missförstånd

"Räknesortering är linjär." Den tar $O(n + k)$, där k är antalet möjliga värden. Här är $k = n^3$: både tid och minne blir $O(n^3)$.

Vanligt missförstånd

"Radixsortering är linjär." Den tar $O(d(n + b))$ för d siffror i bas b . I bas 10 har talen upp till $3 \log_{10} n$ siffror: $O(n \log n)$, ingen vinst.

Idén

Välj basen efter indata: **bas** n . Då har varje tal högst tre siffror, och varje siffra ligger i $[0..n - 1]$: precis lagom för räknesortering.

Uppgift 2: tre omgångar räknasortering

1. Skriv talen i bas n . Alla har högst tre siffror, utom n^3 som skrivs 1000_n : plocka ut alla förekomster av n^3 i en genomgång och lägg dem sist.
2. Fyll på med inledande nollor på tal mindre än n^2 , så att alla får tre siffror.
3. Radixsortera i tre omgångar, **minst signifikanta siffran först**. Varje omgång sorterar n tal efter en siffra i $[0..n-1]$ med räknasortering: $O(n+n) = O(n)$.

Analys

Tre omgångar à $O(n)$, plus $O(n)$ för specialfallet n^3 . Totalt $O(n)$.



Uppgift 2: exempel i bas 4

Med $n = 4$ är basen 4 och talen ligger i $[1..64]$.

Indata

$$45 = 231_4$$

$$7 = 013_4$$

$$30 = 132_4$$

$$18 = 102_4$$

Uppgift 2: exempel i bas 4

Med $n = 4$ är basen 4 och talen ligger i $[1..64]$.

Indata	Omgång 1
	sista siffran
$45 = 231_4$	23 1 (45)
$7 = 013_4$	13 2 (30)
$30 = 132_4$	10 2 (18)
$18 = 102_4$	01 3 (7)

Uppgift 2: exempel i bas 4

Med $n = 4$ är basen 4 och talen ligger i $[1..64]$.

Indata	Omgång 1	Omgång 2
	sista siffran	mitsiffran
$45 = 231_4$	2 3 1 (45)	1 0 2 (18)
$7 = 013_4$	1 3 2 (30)	0 1 3 (7)
$30 = 132_4$	1 0 2 (18)	2 3 1 (45)
$18 = 102_4$	0 1 3 (7)	1 3 2 (30)

Uppgift 2: exempel i bas 4

Med $n = 4$ är basen 4 och talen ligger i $[1..64]$.

Indata	Omgång 1	Omgång 2	Omgång 3
	sista siffran	mitsiffran	första siffran
$45 = 231_4$	2 3 1 (45)	1 0 2 (18)	0 1 3 (7)
$7 = 013_4$	1 3 2 (30)	0 1 3 (7)	1 0 2 (18)
$30 = 132_4$	1 0 2 (18)	2 3 1 (45)	1 3 2 (30)
$18 = 102_4$	0 1 3 (7)	1 3 2 (30)	2 3 1 (45)

Sorterat efter tre omgångar och noll jämförelser mellan talen. I sista omgången har 18 och 30 samma siffra, 1: stabiliteten behåller ordningen från omgång 2, där 18 redan låg före 30.

Uppgift 2: varför stabil, och varför bakifrån?

Vanligt missförstånd

"Vilken sortering som helst duger i varje omgång."

Varje omgång måste vara **stabil**: lika siffror behåller sin inbördes ordning. Annars river omgång 3 det som omgång 1 och 2 byggde upp, och i exemplet kunde 30 hamna före 18. Räknesorteringen från föreläsning 16 är stabil.

Vanligt missförstånd

"Börja med den mest signifikanta siffran, det är ju den som betyder mest."

Då måste varje hink sorteras vidare för sig, rekursivt. Minst signifikant först klarar sig med tre raka genomgångar, just tack vare stabiliteten: sista omgången har sista ordet, och de tidigare avgör bara oavgjorda fall.

Uppgift 2: vad händer om ...?

Talen ligger i	Bas	Omgångar	Tid
$[1..n^3]$	n	3	$O(n)$
$[1..n^c]$, c konstant	n	c	$O(cn) = O(n)$
$[1..n^3]$	10	$\approx 3 \log_{10} n$	$O(n \log n)$
$[1..n^3]$	n^3	1	$O(n^3)$
$[1..2^n]$	n	$\approx n / \log_2 n$	$O(n^2 / \log n)$

- Basen ska vara ungefär lika stor som antalet tal: större bas gör varje omgång dyr, mindre bas ger fler omgångar.
- Och med bitkostnad? $O(n)$ operationer på tal med $O(\log n)$ bitar blir $O(n \log n)$ bitoperationer, och indata är just $\Theta(n \log n)$ bitar. Fortfarande linjärt i indatas storlek.

Uppgift 2: släktingar och historia

Samma idé på andra ställen

- Hinksortering: fördela talen i hinkar efter värde, sortera hinkarna.
- 32-bitars nycklar: fyra omgångar à 8 bitar, alltså bas 256. Grafikkortsbibliotek sorterar heltal ungefär så.

Historia

Holleriths hålkortsmaskiner räknade USA:s folkräkning 1890. Sorterarna som följde tog en kolumn i taget, minst signifikanta först: radixsortering i mekanisk form. Räknesorteringen beskrev Harold Seward för datorer i sitt examensarbete vid MIT 1954.

Tricket

Känt värdeintervall $\Rightarrow$ använd värdena som index i stället för att jämföra dem. Bas n ger ett konstant antal siffror.

Labb 3: teoriredovisning

- Lämna in era skriftliga lösningar till teoriuppgifterna i labb 3.
- Muntlig redovisning i par, som kamratredovisning: den ena startar quizen **Labbateori 3A**, den andra **Labbateori 3B** (Canvas, under Uppgifter). Slumpa vem som tar vilken.
- Ställ frågorna i nummerordning och turas om. Omkring 10 minuter, högst 15.
- Den som frågar bedömer svaret: rätt eller fel. Ingen hjälp på traven, men läs upp rätt svar om det blev fel, och diskutera.
- Minst två rätta svar ger 1 teoripoäng. Läraren jämför med de inlämnade svaren och för in poängen i Canvas.

Sedan: paus

Hjärnan behöver svalna innan uppgift 3 ber den räkna modulo en miljard.

Uppgift 3: Hitta det saknade talet

På en datafil ligger 999 999 999 tal, nämligen alla heltal mellan 1 och 1 000 000 000 utom ett. Vilket tal är det som saknas? Konstruera en algoritm som löser detta problem i konstant minne och linjär tid i en beräkningsmodell som har 32-bitsaritmetik (dvs kan räkna med tal av storlek mindre än $2^{31} = 2\,147\,483\,648$).

Vanligt missförstånd

"Konstant minne? Filen är ju flera gigabyte."

Filen är indata, och den läser vi en gång från början till slut. Konstant minne gäller arbetsminnet: några 32-bitarsvariabler, hur stor filen än är.

Uppgift 3: tre rimliga idéer, tre brutna krav

Idé	Linjär tid	Konstant minne	32 bitar
Sortera och leta efter luckan	×	×	✓
Bitarray: en bit per möjligt tal, 10^9 bitar	✓	× 125 MB	✓
Summera: $x = \frac{n(n+1)}{2} - S'$	✓	✓*	× $\approx 5 \cdot 10^{17}$

*Strikt taget $O(\log n)$ bitar, som Alice påpekar. Här: 59 stycken.

Summan faller på ett enda krav: den är för stor. Kan vi krympa den utan att tappa svaret?

Uppgift 3: samma sak i liten skala, modulo 10

Talen 1 till 10 har summan 55, och $55 \bmod 10 = 5$. Om ett tal x saknas blir summan av resten, modulo 10:

saknat x	1	2	3	4	5	6	7	8	9	10
summan av resten, mod 10	4	3	2	1	0	9	8	7	6	5

- Alla tio kandidater ger olika rester: resten pekar ut x entydigt.
- $x \equiv 5 - (\text{summan av filen}) \pmod{10}$, där resten 0 betyder $x = 10$.
- Det fungerar för att modulen är minst lika stor som antalet kandidater.

Uppgift 3: tillbaka till $n = 10^9$

Summan av alla tal $1..n$ för jämnt n , modulo n :

$$S = \frac{n(n+1)}{2} = \frac{n}{2} \cdot n + \frac{n}{2} \equiv \frac{n}{2} \pmod{n}$$

Låt S' vara summan av talen på filen modulo n . Det saknade talet x uppfyller

$$S \equiv S' + x \pmod{n} \iff x \equiv \frac{n}{2} - S' \pmod{n}$$

Varför ryms det i 32 bitar?

Den löpande summan är alltid $< 10^9$ och nästa tal är högst 10^9 . Deras summa är alltså $< 2 \cdot 10^9 < 2^{31}$, och vi tar modulo innan något svämmar över. Därför just 10^9 : lika många rester som kandidater, och litet nog för 32 bitar.

Vi vet aldrig vad summan är, bara vad den är modulo en miljard. Det räcker, eftersom svaret också är mindre än en miljard. Nästan.

Uppgift 3: algoritmen, och två fällor

```
MissingX(A[1..109-1]) =  
    sum ← 0  
    for ai ∈ A do  
        sum ← (sum + ai) mod 109  
    x ← (500000000 - sum) mod 109  
    if x = 0 then return 109  
    return x
```

Fälla 1: nästan

$10^9 \equiv 0$, så svaret 0 betyder att 10^9 saknas. Summan är bara känd modulo 10^9 , där 0 och 10^9 ser likadana ut.

Fälla 2: negativ modulo

500 000 000 - *sum* kan bli negativt. I C och Java ger $-3 \% 10$ värdet -3 , i Python 7. Lägg till 10^9 först:
 $(500\,000\,000 - \textit{sum} + 10^9) \bmod 10^9$.

Uppgift 3: vad händer om ...?

- **...vi byter + mot XOR?** XOR:a ihop alla tal $1..10^9$ och alla tal på filen. Varje tal som finns med tar ut sig självt, $a \oplus a = 0$, och bara x blir kvar. Alla tal är $< 2^{30}$, så inget kan svämma över, och $1 \oplus 2 \oplus \dots \oplus 10^9 = 10^9$ eftersom $4 \mid 10^9$. Båda fällorna försvinner.
- **...vi byter mot OR eller max?** Då går det inte: de kan inte tas tillbaka. Summa och XOR fungerar för att varje tal kan räknas bort igen.
- **...vi räknar modulo 1000?** Då går x och $x + 1000$ inte att skilja åt. Modulen måste vara minst antalet kandidater.
- **...två tal saknas?** En ekvation, två okända. XOR ger $x \oplus y$: välj en bit där x och y skiljer sig och XOR:a de två halvorna var för sig. Eller summera även kvadraterna.

Uppgift 3: släktingar och historia

Samma idé på andra ställen

- RAID 5: paritetsdisken är XOR av de andra, så en död disk är ett saknat tal.
- Sista siffran i ert personnummer är en kontrollsiffra modulo 10 (Hans Peter Luhn, IBM, 1954).

Historia

Enligt legenden summerade den unge Gauss 1 till 100 på ett ögonblick: 50 par à 101. Historien trycktes först 1856, och Brian Hayes har samlat ett hundratal versioner (2006): detaljerna växer, formeln står sig. Jon Bentleys *Programming Pearls* (1986) har en kusin: hitta ett 32-bitars tal som saknas bland fyra miljarder, med några hundra byte minne.

Tricket

Spara fingeravtrycket, inte datan: summan modulo minst antalet kandidater.

Uppgift 4: Komplex multiplikation

Om man multiplicerar två komplexa tal $a + bi$ och $c + di$ på det vanliga sättet krävs fyra multiplikationer och två additioner av reella tal:

$$(a + bi)(c + di) = (ac - bd) + (ad + bc)i$$

Eftersom multiplikationer är dyrare än additioner (och subtraktioner) lönar det sig att minimera antalet multiplikationer om man ska räkna med stora tal. Hitta på en algoritm som bara använder tre multiplikationer (men fler additioner) för att multiplicera två komplexa tal.

Vanligt missförstånd

"Färre multiplikationer är alltid snabbare." För flyttal i hårdvaran kostar en multiplikation ungefär som en addition, och tricket byter en multiplikation mot tre additioner: förlust. Vinsten kommer med stora tal, där skolmetodens multiplikation kostar $\Theta(n^2)$ och additionen bara $\Theta(n)$.

Uppgift 4: tre multiplikationer, fem additioner

Viggos facit för uppgift 4 lyder i sin helhet: "Egen övning." Vi gör den ändå.

$$A_1 = (a + b)(c - d) = ac - ad + bc - bd \quad A_2 = ad \quad A_3 = bc$$

$$\operatorname{Re}(z) = A_1 + A_2 - A_3 = ac - bd \quad \operatorname{Im}(z) = A_2 + A_3 = ad + bc$$

- Additionerna: $a + b$, $c - d$, $A_1 + A_2$, $-A_3$ och $A_2 + A_3$. Fem stycken.
- Vi byter en multiplikation mot tre additioner och kallar det en besparing. För stora tal är det också en.

Kontroll: $(3 + 2i)(1 + 4i)$

$$A_1 = 5 \cdot (-3) = -15, A_2 = 12, A_3 = 2: \operatorname{Re} = -15 + 12 - 2 = -5, \operatorname{Im} = 12 + 2 = 14.$$

$$\text{Direkt: } 3 + 12i + 2i + 8i^2 = -5 + 14i.$$



Uppgift 4: vad händer om ...?

- **...vi grupperar på andra sätt?** Det finns fler varianter, alla med tre multiplikationer och fem additioner:

$$k_1 = c(a + b), k_2 = a(d - c), k_3 = b(c + d) : \quad \text{Re} = k_1 - k_3, \quad \text{Im} = k_1 + k_2$$

$$P_1 = ac, P_2 = bd, P_3 = (a + b)(c + d) : \quad \text{Re} = P_1 - P_2, \quad \text{Im} = P_3 - P_1 - P_2$$

- **...vi vill klara oss med två?** Går inte. Tre reella multiplikationer är bevisat minimum (följer av Alder och Strassens sats, 1981).
- **...vi räknar med flyttal?** Svaret blir inte bitvis detsamma: $A_1 + A_2 - A_3$ kan ta ut stora tal mot varandra. Numeriska bibliotek har därför "3M"-varianter av komplex matricmultiplikation (t.ex. `zgemm3m`) som ett uttalat val: snabbare, något sämre noggrannhet.
- **...det är polynom?** $(a + bx)(c + dx) = ac + (ad + bc)x + bd x^2$ går på samma sätt med tre multiplikationer. Komplexa tal är polynom där $x^2 = -1$.

Uppgift 4: när tricket körs rekursivt

Dela två n -siffriga heltal på mitten: $x = x_1 B + x_0$ och $y = y_1 B + y_0$. Då är

$$xy = x_1 y_1 B^2 + ((x_1 + x_0)(y_1 + y_0) - x_1 y_1 - x_0 y_0) B + x_0 y_0.$$

Tre multiplikationer av halva storleken i stället för fyra, rekursivt: Karatsubas algoritm.

Rekursion	Lösning (mästarsatsen)	
$T(n) = 4 T(n/2) + O(n)$	$O(n^2)$	skolmetoden
$T(n) = 3 T(n/2) + O(n)$	$O(n^{\log_2 3}) \approx O(n^{1,585})$	Karatsuba 1962
$T(n) = 8 T(n/2) + O(n^2)$	$O(n^3)$	matrismultiplikation
$T(n) = 7 T(n/2) + O(n^2)$	$O(n^{\log_2 7}) \approx O(n^{2,807})$	Strassen 1969

Ensam är en sparad multiplikation en konstant. Rekursivt blir den en ny exponent.

Uppgift 4: har inte en AI slagit Strassen?

Jo, på 4×4 -matriser

2022 hittade DeepMinds AlphaTensor 47 multiplikationer för 4×4 , mot $7^2 = 49$ för Strassen i två nivåer, men bara modulo 2. En vecka senare hade Kauers och Moosbauer i Linz slagit dess 5×5 -rekord. År 2025 hittade AlphaEvolve 48 med komplexa tal, och en månad senare hade Dumas, Pernet och Sedoglavic en rationell variant. Samma trick som i uppgift 4: det nya är vem som letar.

Men inte i uppgift 4

- Två komplexa tal kräver bevisligen tre reella multiplikationer. Ingen sökning hittar två.
- Även 2×2 -matriser är avgjorda: sju är minimum (Winograd 1971). Fynden gäller 4×4 -block, där motståndaren är Strassen i två nivåer.
- Rekursivt ger 48 exponenten $\log_4 48 \approx 2,793$ mot Strassens 2,807 (47 ger 2,777, modulo 2). Rekordet ligger under 2,372, med metoder som aldrig körs i praktiken.

Uppgift 4: historia

Historia

Hösten 1960 gissade Andrej Kolmogorov på ett seminarium i Moskva att multiplikation av n -siffriga tal kräver $\Omega(n^2)$ operationer. Inom en vecka hade studenten Anatolij Karatsuba, 23 år, motbevisat gissningen. Kolmogorov skrev själv artikeln (1962) i Karatsubas och Ofmans namn; Karatsuba fick veta det när särtrycken kom.

Och sedan

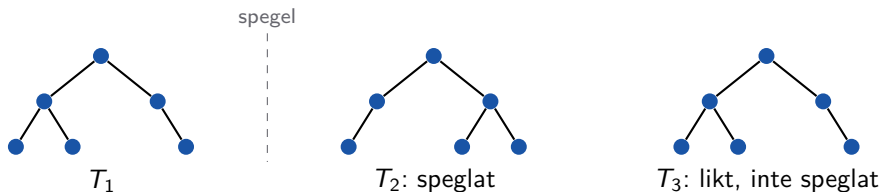
Tricket med tre multiplikationer för komplexa tal brukar tillskrivas Gauss. Strassen gjorde samma sak med matriser 1969: sju multiplikationer i stället för åtta. År 2019 visade Harvey och van der Hoeven att heltal kan multipliceras i $O(n \log n)$, vilket troligen är optimalt.

Tricket

Byt en dyr operation mot flera billiga: tre multiplikationer och fem additioner i stället för fyra och två.

Uppgift 5: Binärträd med speglad struktur

Två binära träd sägs ha *speglad struktur* om det ena är en spegelbild av det andra, det vill säga att om man byter vänster och höger överallt i det ena trädet så blir träden strukturekvivalenta. Konstruera och analysera tidskomplexiteten för en effektiv dekompositionsalgoritm som avgör ifall två binärträd har speglad struktur.



Noderna har inget innehåll: det är bara formen som räknas.

Uppgift 5: vänster mot vänster är fel spegel

Vanligt missförstånd

"Jämför vänster med vänster och höger med höger." Det testas om träden är *lika*. T_1 och T_3 klarar det testet, T_1 och T_2 gör det inte.

```
StrukturLika( $T_1$ ,  $T_2$ ) =  
  if  $T_1$  = NIL or  $T_2$  = NIL then  
    return  $T_1$  = NIL and  $T_2$  = NIL  
  return StrukturLika( $T_1$ .left,  $T_2$ .left)  
    and StrukturLika( $T_1$ .right,  $T_2$ .right)
```

```
SpegladeTräd( $T_1$ ,  $T_2$ ) =  
  if  $T_1$  = NIL or  $T_2$  = NIL then  
    return  $T_1$  = NIL and  $T_2$  = NIL  
  return SpegladeTräd( $T_1$ .left,  $T_2$ .right)  
    and SpegladeTräd( $T_1$ .right,  $T_2$ .left)
```

Två algoritmer, en enda skillnad: vilka delträd som paras ihop.

Vanligt missförstånd

"Jämför traverseringsordningarna." Utan innehåll i noderna ger en inorder bara antalet noder. Med NIL-markörer går det, till priset av $O(n)$ extra minne.

Uppgift 5: lösningen

Dekomposition: stega ner i båda träden samtidigt, med korsade pekare. Vänstra delträdet i T_1 ska spegla högra delträdet i T_2 , och tvärtom. Basfall: minst ett av träden är tomt, och då är de spegelbilder bara om båda är tomma.

```
SpegladeTräd( $T_1$ ,  $T_2$ ) =  
  if  $T_1$  = NIL or  $T_2$  = NIL then  
    return  $T_1$  = NIL and  $T_2$  = NIL  
  return SpegladeTräd( $T_1$ .left,  $T_2$ .right)  
    and SpegladeTräd( $T_1$ .right,  $T_2$ .left)
```

Korrekthet: strukturell induktion

Basfallet stämmer: ett tomt träd speglar bara ett tomt träd. Induktionssteget stämmer: om delträden speglar varandra korsvis så speglar träden varandra.

Uppgift 5: tid, och vad händer om ...?

Tidskomplexitet

Högst ett anrop per delträd och konstant arbete per anrop: $O(n)$. Noggrannare $O(\min(n_1, n_2))$, eftersom rekursionen bara går vidare där båda träden har en nod. Rekursionsdjupet är höjden, upp till n för ett snett träd: det enda som växer är stacken.

- ...ett träd ska vara symmetriskt? *SpeglandeTräd($T.left$, $T.right$)*.
- ...trädet ska speglas? Byt vänster och höger i varje nod, rekursivt: samma genomgång, med en tilldelning i stället för en jämförelse.
- ...noderna har innehåll? Kräv även $T_1.key = T_2.key$.
- ...träden inte är binära? Jämför barnlistorna i omvänd ordning.

Uppgift 5: historia

Historia

År 2015 twittrade Max Howell, som skrivit pakethanteraren Homebrew, att Google tackat nej till honom: nio av tio av deras ingenjörer använde hans program, men han kunde inte invertera ett binärträd på whiteboarden. Invertera är exakt "spegla" från förra bilden. Några rader, och nu kan ni dem.

Beviset

Strukturell induktion som metod för att bevisa egenskaper hos program formulerades av Rod Burstall 1969. Det är vanlig induktion, fast över trädets form i stället för över de naturliga talen.

Tricket

Dekomposition med korsade pekare: vänster mot höger, höger mot vänster.

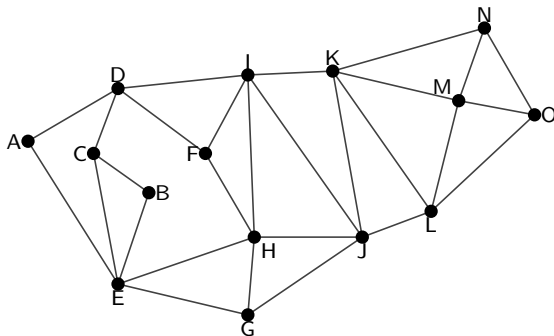
Uppgift 6: Partyproblemet

Indata är en lista med n personer, ett heltal k och en lista med vilka personer som känner varandra. Du vill bjuda så många av personerna som möjligt på ditt party, men för att alla ska trivas vill du att varje inbjuden gäst ska känna minst k av dom övriga gästerna. Konstruera och analysera en algoritm som löser detta problem i linjär tid i indatas storlek.

Modell

En graf $G = (V, E)$: personer är hörn och bekantskaper är kanter, m stycken, lagrade i kantlistor. Sökt: den största inducerade delgrafen där varje hörn har gradtal minst k .

Uppgift 6: exempel med $k = 3$

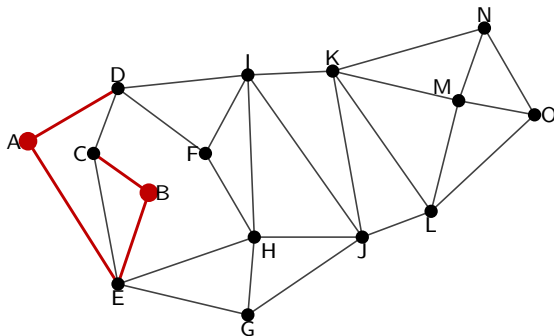


15 personer, $k = 3$.

Vanligt missförstånd

"Bjud alla som känner minst 3."
Det blir 13 gäster, men då känner C bara 2 av dem.

Uppgift 6: exempel med $k = 3$

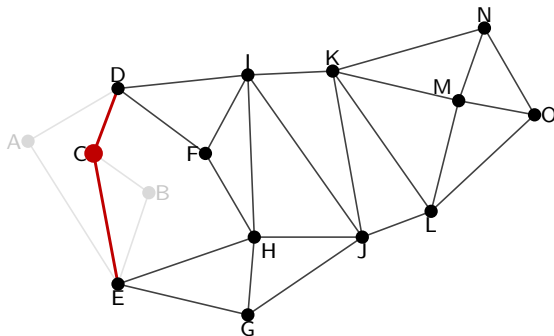


A och B känner färre än 3. De kan aldrig komma.

Kö: A, B

Gäster kvar: 13

Uppgift 6: exempel med $k = 3$

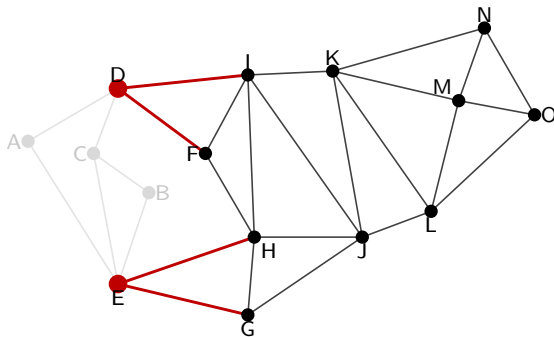


Utan B sjunker C till 2.

Kö: C

Gäster kvar: 12

Uppgift 6: exempel med $k = 3$

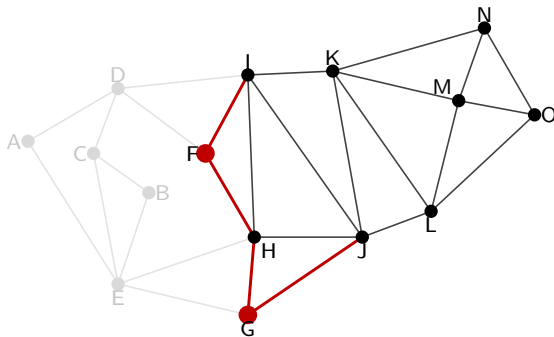


Utan C sjunker D och E till 2.

Kö: D, E

Gäster kvar: 10

Uppgift 6: exempel med $k = 3$

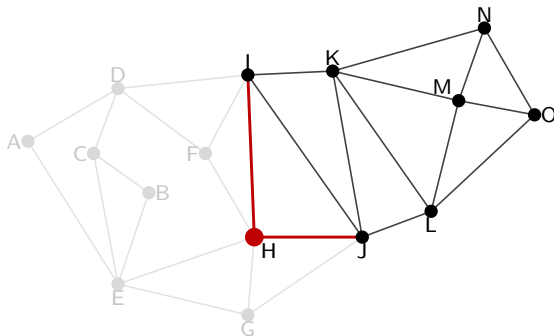


Utan D och E sjunker F och G till 2.

Kö: F, G

Gäster kvar: 8

Uppgift 6: exempel med $k = 3$

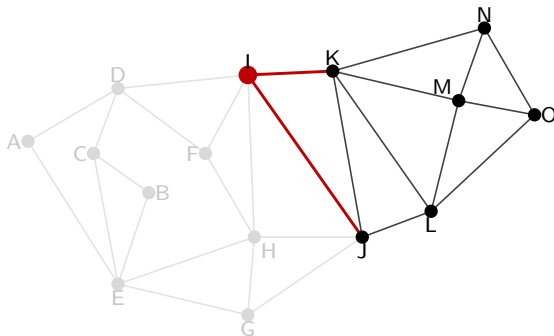


Utan F och G sjunker H till 2.

Kö: H

Gäster kvar: 7

Uppgift 6: exempel med $k = 3$

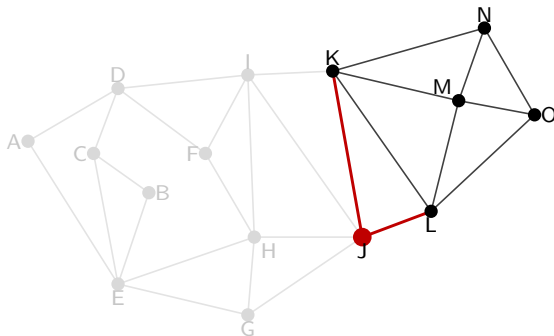


Utan H sjunker I till 2.

Kö: 1

Gäster kvar: 6

Uppgift 6: exempel med $k = 3$

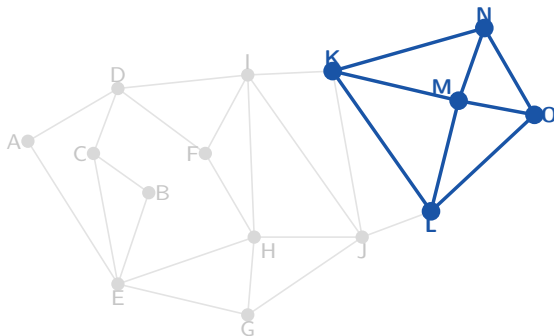


Utan I sjunker J till 2.

Kö: J

Gäster kvar: 5

Uppgift 6: exempel med $k = 3$



Utan J sjunker K och L till 3, och 3 räcker.

Kö: tom

Gäster kvar: 5

K, L, M, N och O känner minst 3 var. Två personer med för få bekanta välte åtta till.

Uppgift 6: algoritmen

```
1  foreach x ∈ V do
2      if dx < k then Q.Put(x)
3  while not Q.Empty() do
4      x ← Q.Get()
5      foreach (x, v) ∈ x.kantlista do
6          if dv ≥ k then
7              dv ← dv - 1
8              if dv < k then Q.Put(v)
9  write 'Lösningen består av:'
10 foreach x ∈ V do
11     if dx ≥ k then write x
```

Vanligt missförstånd

"Villkoret på rad 6 är bara en optimering."

Utan det läggs redan köade hörn in igen. I exemplet sänker B C, sedan sänker C B, och så vidare: B och C skickar varandra fram och tillbaka i kön för evigt. Algoritmen terminerar inte.

Villkoret är en "besökt"-markering, precis som i bredden först-sökning.

Uppgift 6: korrekthet och tidskomplexitet

Varje utskriven gäst trivs

Varje struket hörn drar av sig från sina grannar som ännu är kvar. När kön är tom är d_x därför, för varje hörn som aldrig köats, exakt antalet grannar som också är kvar, och x skrivs ut bara om $d_x \geq k$.

Ingen som strukits kunde ha varit med

Induktion över köordningen: när v köas känner v färre än k av dem som inte redan köats, och de redan köade kan enligt induktionsantagandet inte vara med i någon giltig gästlista. Alltså kan inte v det heller.

Svaret innehåller därför varje giltig gästlista: det är den största, och den är unik.

Tidskomplexitet

Varje hörn köas högst en gång (rad 6) och behandlas ett konstant antal gånger. Varje kant ses högst två gånger, en gång från varje ände. Totalt $O(n + m)$: linjärt i indatas storlek.

Uppgift 6: två missförstånd till

Vanligt missförstånd

"Stryk alla med för låg grad, gå igenom listan igen, och upprepa tills inget ändras."

Korrekt, men varje varv kostar $O(n + m)$, och en kaskad kan kräva $\Theta(n)$ varv: en lång kedja som går igenom i fel riktning tappar en person per varv. Totalt $\Theta(n(n + m))$.
Kön gör att vi bara tittar där något just har ändrats.

Vanligt missförstånd

"Man måste stryka den med lägst grad först."

Nej. Svaret är unikt, så ordningen spelar ingen roll: kön får ta dem som de kommer.

Uppgift 6: vad händer om vi byter ett ord?

Varje gäst ska känna ...	Problemet heter	Svårighet
... minst k av de andra	k -kärna	$O(n + m)$, idag
... alla de andra	största klick	NP-svårt
... ingen av de andra	största oberoende mängd	NP-svårt

- Varför är "minst k " så snällt? Gästlistan kan bara krympa, och den som känner för få nu känner inte fler senare. Ett nej blir aldrig ett ja.
- För "alla" och "ingen" finns ingen känd algoritm i polynomisk tid. Klick var ett av Karps 21 NP-fullständiga problem 1972. Mer om det i kursens sista del.

Uppgift 6: släktingar och historia

Samma idé på andra ställen

- Topologisk sortering (Kahn 1962): köa hörnen vars ingrad når 0 och minska grannarnas räknare. Byt "når 0" mot "understiger k ", så har ni partyt.
- Degenereringsordning (Matula och Beck 1983): stryk upprepat lägsta graden.
- Batagelj och Zaveršnik (2003): kärnorna för alla k på en gång, i $O(m)$.

Historia

Stephen Seidman införde k -kärnor 1983, i tidskriften *Social Networks*. Partyproblemet är alltså inte ett skämt, det är sociologi.

Tricket

Stryk den som omöjligt kan vara med, och låt en kö hålla reda på kaskaden.

Känner du igen tricket?

Problemet

Bland $n + 1$ tal i $[1..n]$ förekommer exakt ett tal två gånger. Hitta det i konstant minne.

Sortera tio miljoner IPv4-adresser.

Beräkna $(a + bx)(c + dx)$ med tre multiplikationer.

Är ett binärträd symmetriskt kring roten?

I vilken ordning kan kurserna läsas, givet förkunskapskraven?

Ligger drönaren i den konvexa flygzonen? En miljon frågor i sekunder.

Känner du igen tricket?

Problemet	Tricket
Bland $n + 1$ tal i $[1..n]$ förekommer exakt ett tal två gånger. Hitta det i konstant minne.	Uppgift 3: summan, fast baklänges
Sortera tio miljoner IPv4-adresser.	Uppgift 2: radix, fyra omgångar à 8 bitar
Beräkna $(a + bx)(c + dx)$ med tre multiplikationer.	Uppgift 4: komplexa tal är polynom med $x^2 = -1$
Är ett binärträd symmetriskt kring roten?	Uppgift 5: <i>SpeglandeTräd</i> ($T.left$, $T.right$)
I vilken ordning kan kurserna läsas, givet förkunskapskraven?	Uppgift 6: räknare och kö, som hos Kahn
Ligger drönaren i den konvexa flygzonen? En miljon frågor i sekunder.	Uppgift 1: binärsök i solfjädern

Kravet	Ledtråden i texten
1 $O(\log n)$	"konvex": hörnen ligger sorterade efter vinkel runt p_1
2 $O(n)$	" $[1..n^3]$ ": tre siffror i bas n
3 32 bitar	"alla heltal mellan 1 och 10^9 utom ett": räkna modulo 10^9
4 tre multiplikationer	"(men fler additioner)": står rakt ut
5 två träd på en gång	"dekompositionsalgoritm": korsade pekare
6 linjär tid	"minst k ": ett nej blir aldrig ett ja

Viggo skriver alltid ut ledtråden. Han kallar den bara för ett krav.

Vem gjorde vad

- **Viggo Kann**: uppgifterna, lösningsförslagen, pseudokoden och korrekthetsbevisen. Utom uppgift 4, där facit säger "Egen övning".
- **Anton Grensjö** (2017): kryssprodukten, teckentest, frågan om den undre gränsen, minst signifikant siffra först, bitarrayen och summan som mellansteg, XOR-varianten.
- **Lisa Li** (2019): stråle först och dekomposition sedan, den undre gränsen och grafmodellen, i kompakt form.
- **Alice Heavy** (2020): de stegvisa polygonbilderna, ledningsfrågan om logaritmer, modulo 10-exemplet, fällorna med 0 och negativ modulo, partykaskaden med $k = 3$ och kamratredovisningen.

Missförstånden, varianterna, släktingarna och historien är nya för i år. Eventuella fel i dem är mina.

Källor till historien

- C. Jordan, *Cours d'analyse*, 1887. O. Veblen, Theory on plane curves in non-metrical analysis situs, *Trans. AMS* 6, 1905.
- M. Shmrat, Algorithm 112: Position of point relative to polygon, *CACM* 5(8), 1962.
- R. Graham, An efficient algorithm for determining the convex hull of a finite planar set, *Inf. Proc. Letters* 1, 1972.
- J. Bentley, *Programming Pearls*, Addison-Wesley, 1986.
- J. Bloch, Nearly all binary searches and mergesorts are broken, Google Research Blog, 2006.
- H. H. Seward, Information sorting in the application of electronic digital computers to business operations, examensarbete, MIT, 1954.
- W. Sartorius von Waltershausen, *Gauss zum Gedächtniss*, 1856.
- B. Hayes, Gauss's day of reckoning, *American Scientist* 94(3), 2006.
- H. P. Luhn, Computer for verifying numbers, US-patent 2950 048, ansökt 1954, beviljat 1960.
- R. Karp, Reducibility among combinatorial problems, 1972.
- S. Winograd, On multiplication of 2×2 matrices, *Linear Algebra Appl.* 4, 1971.
- A. Fawzi m.fl., Discovering faster matrix multiplication algorithms with reinforcement learning, *Nature* 610, 2022.
- A. Karatsuba, Yu. Ofman, *Doklady Akad. Nauk SSSR* 145, 1962.
- A. Karatsuba, The complexity of computations, *Proc. Steklov Inst.* 211, 1995.
- V. Strassen, Gaussian elimination is not optimal, *Numer. Math.* 13, 1969.
- A. Alder, V. Strassen, On the algorithmic complexity of associative algebras, *TCS* 15, 1981.
- D. Harvey, J. van der Hoeven, Integer multiplication in time $O(n \log n)$, *Annals of Math.* 193, 2021.
- R. M. Burstall, Proving properties of programs by structural induction, *Computer Journal* 12(1), 1969.
- M. Kauers, J. Moosbauer, The FBHHRBNRSSSHK-algorithm for multiplication of 5×5 matrices is still not the end of the story, arXiv, 2022.
- Google DeepMind, AlphaEvolve: a Gemini-powered coding agent for designing advanced algorithms, blogg, 14 maj 2025.
- J.-G. Dumas, C. Pernet, A. Sedoglavic, A non-commutative algorithm for multiplying 4×4 matrices using 48 non-complex multiplications, arXiv:2506.13242, 2025.
- M. Howell, inlägg på Twitter, juni 2015.
- S. B. Seidman, Network structure and minimum degree, *Social Networks* 5(3), 1983.
- A. B. Kahn, Topological sorting of large networks, *CACM* 5(11), 1962.
- D. Matula, L. Beck, Smallest-last ordering and clustering and graph coloring algorithms, *JACM* 30(3), 1983.
- V. Batagelj, M. Zaveršnik, An $O(m)$ algorithm for cores decomposition of networks, arXiv cs/0310049, 2003.

Innehåll, som avslutning

Dagens tema

Uppgift 1: Inuti eller utanför?

Uppgift 2: Sortering av små heltal

Labb 3: teoriredovisning

Uppgift 3: Hitta det saknade talet

Uppgift 4: Komplex multiplikation

Uppgift 5: Binärträd med speglad struktur

Uppgift 6: Partyproblemet

Avslutning